ChatGPT

60-Day Learning Challenge Roadmap

Phase 1: ROS2 Foundations & Robot Simulation (Days 1–15)

Week 1 (Days 1–7): Core ROS2 Programming

- **Day 1: ROS2 Jazzy workspace & first package.** Install ROS2 Jazzy on Ubuntu 24.04 (officially supported [1] [2]) and create a new `ros2_ws` workspace. Follow the ROS2 "Creating a workspace" tutorial [3]. Verify `colcon build` works and write your first minimal Python node. (Recommended course: *"ROS 2 for Beginners (ROS Jazzy – 2026)"* on Udemy for step-by-step guidance.)

- **Day 2: ROS2 Nodes & Topics (Python).** Write a simple publisher node in Python. Use the official ROS2 tutorial "Writing a simple publisher and subscriber (Python)" [4] to create a talker node that publishes on a topic. Confirm you can subscribe with `ros2 topic echo`.

- **Day 3: ROS2 Topics & Subscribers (C++).** Write a corresponding listener node in C++. Use the ROS2 C++ example in the official docs [4] (adjusted to C++). The Udemy course *"ROS 2 Jazzy: Complete Robotics Developer Course"* also covers topics and nodes in both Python and C++.

- **Day 4: Services & Clients.** Learn ROS2 services. Follow the "Writing a simple service and client (Python)" tutorial [5]: create a service node (e.g. add two ints) and a client node that requests it. Test with `ros2 run py_srvcli service` and `ros2 run py_srvcli client`. (For a quick intro video, search YouTube for "ROS2 services tutorial" or see the official tutorial with sample code [5].)

- **Day 5: Actions (Long-running tasks).** Explore ROS2 actions. Use the "Writing an action server and client (Python)" tutorial [6] (ROS2 Intermediate). Create a Fibonacci action server and client. Try sending a goal via `ros2 action send_goal`. (If stuck, look up a YouTube walkthrough on ROS2 actions, or the Udemy *"ROS2 Level 2"* course which introduces actions.)

- **Day 6: TF2 Transforms & Coordinate Frames.** Learn TF2 basics. Read the ROS2 TF2 primer (e.g. "Introducing tf2" [7]). Practice creating a static transform broadcaster and listener in Python. Verify frame relationships in RViz2. The Udemy course *"ROS2 Level 2 (TF/URDF/RViz/Gazebo)"* covers TF2 and frames (see its intro).

- **Day 7: Launch files & Parameters.** Practice writing ROS2 launch files. Follow the "Creating a launch file" tutorial (ROS2 Intermediate) and "Using parameters" tutorials [8] [5]. Write an XML or Python launch file that starts multiple nodes with remapped topics and YAML parameters. (The *Udemy ROS2 for Beginners* course also includes examples of launch files and YAML configs.)

Week 2–3 (Days 8–15): Simulation, SLAM & Navigation

- **Days 8–9: URDF & Robot Model.** Design a simple URDF robot. Follow the official URDF tutorials [7] or use a template (e.g. two-wheel robot with a sensor). Visualize your URDF in RViz2 using `robot_state_publisher`. (The *ROS2 Level 2* Udemy course has step-by-step guides for creating URDF links and joints [9].) Make sure to add Gazebo `<collision>` and `<visual>` elements.

- **Day 10: Gazebo Simulation.** Run Gazebo Harmonic (supported on Ubuntu 24.04 [10]) and spawn your URDF robot. Use the `gazebo_ros` spawn script or `gz sim`. Attach simple sensor plugins (e.g. laser or camera) via your URDF. If needed, follow the Gazebo setup guide. (The *ROS2 Level 2* course's Gazebo section [11] shows spawning a robot with sensors in Gazebo.)

- **Day 11: SLAM with LiDAR.** Use ROS2's SLAM Toolbox to build a map. Run a 2D LiDAR sensor (in simulation) and launch `slam_toolbox`. Experiment with teleoperating the robot (via `ros2 topic pub` or RViz teleop) and saving the map. See the ROS2 Navigation tutorials for SLAM setup.

- **Days 12–13: Nav2 Navigation Stack.** Configure the Nav2 stack. Follow the ROS2 Navigation tutorial (e.g. "Navigation2 tutorials" on docs.ros.org) to set up costmaps, planners (DWB/SLAM), and 2D navigation. Test autonomous path planning: publish a `PoseStamped` goal in RViz and let the simulated robot navigate. (The Udemy *"ROS2 Level 2"* course includes Nav2 examples.)

- **Days 14–15: Phase 1 Capstone.** Combine SLAM and Nav2: have your Gazebo robot autonomously map an environment and then navigate to a target. Document the process and results. (Use RViz2 to visualize map, robot pose, and path.)

Phase 2: Computer Vision for Robotics (Days 16–30)

Days 16–22: Vision Pipeline in ROS2

- **Day 16: ROS2 Image Topics & CvBridge.** Publish camera images in ROS2. Use your laptop's webcam or Gazebo camera and the `v4l2_camera` or `usb_cam` ROS2 driver. In a Python node, subscribe to the image topic and convert to OpenCV (`CvBridge`). (Example: modify the ROS1 cv_bridge example to ROS2, or follow a ROS2 OpenCV tutorial on GitHub.)

- **Days 17–18: YOLOv8 Detection Node.** Set up YOLOv8 (Ultralytics) for object detection. Install the Ultralytics package in Python. Write a ROS2 node that takes camera images, runs YOLOv8 inference, and publishes bounding boxes. For guidance, see Ultralytics YOLO docs or a YouTube tutorial on YOLO+ROS2 [12]. (Optionally, wrap YOLO in a ROS2 Python node publishing a custom `BoundingBoxes2D` message.)

- **Days 19–20: Object Tracking (SORT).** Implement a simple object tracker. Use the SORT algorithm or OpenCV trackers. Feed YOLO detections into SORT to maintain object IDs across frames. Publish tracking info as ROS2 messages (e.g. `/tracked_objects`). (Resources: SORT GitHub and ROS2 examples of tracking nodes.)

- **Day 21: ArUco Marker Detection.** Use OpenCV's ArUco module to detect fiducial markers in images. In a ROS2 node, subscribe to camera images and detect markers. Compute marker poses and publish them (e.g. as PoseStamped). (Tutorial: OpenCV ArUco guide.) This helps with robot localization.

- **Day 22: Mini-project:** Build a ROS2 node that announces detections. For example, when YOLO or ArUco sees an object, have it trigger a voice synthesis (e.g. using `sound_play` or a simple `espeak` call). Test by writing a node that listens for detection messages and runs text-to-speech.

Days 23–30: Depth & Visual Control

- **Days 23–24: Depth Perception.** Experiment with depth. Use MiDaS (monocular depth) or a stereo camera in simulation. In Python, run MiDaS on incoming images to estimate depth. (See the MiDaS GitHub for inference code.) Alternatively, spawn a stereo camera in Gazebo and convert disparity to depth in ROS2 (PCL filters).

- **Day 25: Point Cloud Basics.** Publish a ROS2 `PointCloud2` (from Gazebo depth camera or LiDAR). Use the PCL library in a node to filter/segment the cloud (e.g. downsampling or clustering). ROS2 PCL tutorials can help. (The REP-2000 doc [13] notes Gazebo Harmonic works with ROS2.)

- **Days 26–27: Visual Servoing.** Implement a basic visual servo: e.g. move the robot toward an object until it's centered in the camera view. Write a control loop node: use bounding box center (from YOLO) and send velocity commands (`/cmd_vel`) to center the object. Test in Gazebo (e.g. having the robot approach a detected red ball).

- **Days 28–30: Phase 2 Capstone.** Create a Gazebo demo where your robot finds and follows a target object (e.g. a colored cube) using YOLOv8, and navigates to it. Integrate your vision node with Nav2: the robot localizes via SLAM, detects the object, and plans a goal near it. Record a demo, and update your GitHub with code.

Phase 3: Edge AI & Embedded Intelligence (Days 31–45)

Days 31–38: Model Compression & Edge Deployment

- **Days 31–32: TensorFlow Lite (TFLite).** Learn TFLite conversion. Take a small image model (e.g. ResNet or custom CNN in PyTorch). Convert it to TensorFlow and then to TFLite format (`.tflite`). Measure inference time on desktop. See TensorFlow Lite guides. (Example: use the `tflite_convert` CLI or Python API.)

- **Day 33: ONNX Runtime.** Export a model to ONNX and run it with ONNX Runtime in Python. Compare performance to TFLite. Use the ONNX Runtime tutorials [14] for sample code. (ONNX site has docs on quantization [15], which is useful for next step.)

- **Day 34: Model Optimization.** Apply INT8 quantization to your TFLite model (post-training quantization). Also try pruning. The Udemy *"Edge AI & TinyML"* course covers these methods (model compression) [16]. Verify the model is ~4–8× smaller. Test accuracy drop, if any.

- **Days 35–36: Edge Impulse on ESP32.** Use Edge Impulse to build a small classifier (e.g. hand gesture, accelerometer motion). Sign up at Edge Impulse, collect sample data (or use provided datasets), train a TinyML model in the Edge Impulse studio. Deploy to ESP32: follow the Edge Impulse docs for ESP32 [17] [18] (connect device, run `edge-impulse-daemon`, etc.). Flash the model to ESP32 and test locally.

- **Days 37–38: TFLite for Microcontrollers (ESP32).** Run a tiny TFLite model on the ESP32. Use the TensorFlow Lite Micro library. For example, convert a simple sine-model or digit model to a C array and load it on ESP32. The OpenELAB guide provides step-by-step instructions for ESP32 and TFLite [19]. (Resource: *"TensorFlow Lite On ESP32"* tutorial [20].)

Days 39–45: micro-ROS & Hardware Integration

- **Days 39–40: micro-ROS Setup.** Turn the ESP32 into a ROS2 node. Follow a micro-ROS tutorial (e.g. Hackster's "micro-ros on ESP32 using Arduino IDE" [21]). Install the micro-ROS Agent on Ubuntu, and upload a micro-ROS sketch to ESP32. Confirm ESP32 publishes sensor data (e.g. IMU) to ROS2 topics.

- **Day 41: ESP32 → ROS2 Sensors.** Wire up an IMU (MPU6050) or ultrasonic distance sensor (HC-SR04) to ESP32. In your micro-ROS firmware, publish readings as ROS2 messages (e.g. `sensor_msgs/Imu`). Use the Hackster tutorial [22] as guidance for connecting and publishing. Verify topics appear on your PC.

- **Days 42–43: ROS2 → ESP32 Actuators.** Send commands from ROS2 to ESP32. For example, attach an LED or servo to ESP32. Write a micro-ROS subscriber that listens for a custom topic (e.g. `/led_cmd`) and toggles the LED or moves the servo. Test by publishing messages from PC to `/led_cmd`.

- **Days 44–45: Embedded Capstone.** Combine your micro-ROS and AI work: have the ESP32 run a gesture classifier (from Edge Impulse or TFLite) and publish detected gestures to ROS2. In Gazebo, make the simulated robot react: e.g. hand gesture "forward" makes robot drive. Demonstrate this loop: vision node → ROS2 topics → Nav2. Document the integration.

Phase 4: Final Project "VisioBot" & Showcase (Days 46–60)

- **Days 46–48: Architecture Design.** Plan the final system. Draw an architecture diagram showing ROS2 nodes: camera/image processing (YOLO), SLAM/Nav2, and micro-ROS bridge. Sketch how topics and actions flow. Break tasks into packages (e.g. `visbot_vision`, `visbot_navigation`, `visbot_hardware`).

- **Days 49–52: Build & Integrate.** Write the remaining code: merge your YOLO detection node with the Nav2 goal-sender, and connect the ESP32 command node. Ensure all nodes communicate via ROS2. Test incremental integration in Gazebo: e.g., vision node correctly detects objects; Nav2 receives goals; ESP32 commands have effect. Use RViz2 to visualize bounding boxes, map, and robot pose.

- **Days 53–55: Testing & Debugging.** Thoroughly test the VisioBot. Simulate scenarios: object detection, navigation around obstacles, and response to ESP32 gestures. Debug any issues, adjust parameters. Profile performance (FPS of vision, SLAM mapping rate). Finalize RViz2 dashboards for demonstration (e.g. visualization of camera feed + detection overlay).

- **Days 56–57: Demo Recording & Documentation.** Record a demo video of the Gazebo VisioBot: show it detecting objects with YOLOv8, navigating via SLAM, and responding to a hand gesture from a phone or remote (simulating ESP32). Update your GitHub with a comprehensive README: include the system diagram, setup instructions, and code links.

- **Days 58–59: Outreach & Portfolio.** Write a LinkedIn article and short blog post summarizing your 60-day journey and key learnings. Update your portfolio site and GitHub profile with links to the project. Craft tweets for each phase's highlight.

- **Day 60: Final Showcase Post.** Publish a wrap-up tweet (or X post) tagging @lftechnology, including your demo video and a reflective thread on your learning. Highlight how this project ties together ROS2, computer vision, and embedded AI (citing technologies like ROS2 Jazzy [1], YOLOv8, micro-ROS [21], and TFLite [19]).

Each day's topic above is supported by official tutorials and up-to-date courses. For example, ROS2's Jazzy docs cover workspace, packages, nodes, services, and actions [3] [5] [6]. Gazebo Harmonic on Ubuntu 24.04 is officially supported [10] [1]. For Edge AI, the TFLite-on-ESP32 guide [20] and Edge Impulse docs [18] provide practical steps. The cited Udemy and learning resources (e.g. *"ROS2 for Beginners"* [23], *"Edge AI & TinyML"* [16]) will help you master each day's content. This plan emphasizes hands-on simulation first, then light hardware, ensuring you can progress even without a 3D printer or full robot kit.

Sources: Official ROS 2 Jazzy tutorials [3] [24] [4] [5] [6], ROS 2 platform support [1] [2]; Gazebo docs [10]; TFLite/Edge Impulse guides [20] [18]; Udemy courses and tutorials [23] [16].

---

[1] [13] ROS 2 Releases and Target Platforms | Robotics Enhancement Proposals
https://reps.openrobotics.org/rep-2000/

[2] Ubuntu (deb packages) — ROS 2 Documentation: Jazzy documentation
https://docs.ros.org/en/jazzy/Installation/Ubuntu-Install-Debs.html

[3] [7] [8] Creating a workspace — ROS 2 Documentation: Jazzy documentation
https://docs.ros.org/en/jazzy/Tutorials/Beginner-Client-Libraries/Creating-A-Workspace/Creating-A-Workspace.html

[4] Writing a simple publisher and subscriber (Python) — ROS 2 Documentation: Jazzy documentation
https://docs.ros.org/en/jazzy/Tutorials/Beginner-Client-Libraries/Writing-A-Simple-Py-Publisher-And-Subscriber.html

[5] Writing a simple service and client (Python) — ROS 2 Documentation: Jazzy documentation
https://docs.ros.org/en/jazzy/Tutorials/Beginner-Client-Libraries/Writing-A-Simple-Py-Service-And-Client.html

[6] Writing an action server and client (Python) — ROS 2 Documentation: Jazzy documentation
https://docs.ros.org/en/jazzy/Tutorials/Intermediate/Writing-an-Action-Server-Client/Py.html

9 11 23 ROS 2 for Beginners Level 2 - TF | URDF | RViz | Gazebo
https://www.udemy.com/course/ros2-tf-urdf-rviz-gazebo/?srsltid=AfmBOopJ4facbwL-bwnCI_mWmJXUNONiLjJzkXEUDBTaXd0ZqAhb83EE

10 Binary Installation on Ubuntu — Gazebo jetty documentation
https://gazebosim.org/docs/latest/install_ubuntu/

12 How to Use YOLOv8 with ROS2 - YouTube
https://www.youtube.com/watch?v=XqibXP4lwgA

14 15 Tutorials | onnxruntime
https://onnxruntime.ai/docs/tutorials/

16 Practical Guide to Edge AI and TinyML for Smarter Devices
https://www.udemy.com/course/practical-guide-to-edge-ai-and-tinyml-for-smarter-devices/?srsltid=AfmBOorENE_GEBNA3ChIbTAXI6kwzMZ4L-yNjELtmpb9guZU3QAs6JPi

17 18 Espressif ESP-EYE - Edge Impulse Documentation
https://docs.edgeimpulse.com/hardware/boards/espressif-esp32

19 20 TensorFlow Lite On ESP32 – OpenELAB Technology Ltd.
https://openelab.io/blogs/learn/tensorflow-lite-on-esp32?srsltid=AfmBOopi53b-w65ZODp9VzzN4iiGMg8Z8IFOvRLOSv8_XTQxrAP__twf

21 22 Micro ros on ESP32 using arduino IDE - Hackster.io
https://www.hackster.io/514301/micro-ros-on-esp32-using-arduino-ide-1360ca

24 Creating a package — ROS 2 Documentation: Jazzy documentation
https://docs.ros.org/en/jazzy/Tutorials/Beginner-Client-Libraries/Creating-Your-First-ROS2-Package.html